

Java String Operations

A practical guide to commonly used String methods in Java, with explanations, code examples, and sample outputs.

1. What is a String?

In Java, a String is an object used to store a sequence of characters. String objects are immutable, which means that once a String is created, its original contents cannot be changed. Operations such as `toUpperCase()` or `replace()` return a new String.

```
String name = "Srinivas";  
System.out.println(name);
```

Output: Srinivas

2. length()

Returns the number of characters in the String.

```
String s = "Hello";  
System.out.println(s.length());
```

Output: 5

3. charAt()

Returns the character at the specified index. Indexing starts from 0.

```
String s = "Java";  
System.out.println(s.charAt(1));
```

Output: a

4. substring()

Returns part of a String. `substring(start, end)` includes start but excludes end.

```
String s = "Programming";  
System.out.println(s.substring(0, 7));
```

Output: Program

5. equals()

Compares two Strings by their actual contents. It is case-sensitive.

```
String a = "Java";  
String b = "Java";  
System.out.println(a.equals(b));
```

Output: true

6. equalsIgnoreCase()

Compares two Strings while ignoring differences in uppercase and lowercase.

```
String a = "Java";  
String b = "JAVA";  
System.out.println(a.equalsIgnoreCase(b));
```

Output: true

7. compareTo()

Compares two Strings lexicographically. It returns 0 when both are equal, a negative value when the first String comes before the second, and a positive value when it comes after.

```
String a = "Apple";  
String b = "Banana";  
System.out.println(a.compareTo(b));
```

Output: -1

8. toUpperCase()

Converts all alphabetic characters to uppercase.

```
String s = "hello java";  
System.out.println(s.toUpperCase());
```

Output: HELLO JAVA

9. toLowerCase()

Converts all alphabetic characters to lowercase.

```
String s = "HELLO JAVA";  
System.out.println(s.toLowerCase());
```

Output: hello java

10. trim()

Removes leading and trailing whitespace from a String.

```
String s = "  Java  ";  
System.out.println(s.trim());
```

Output: Java

11. contains()

Checks whether the String contains a specified sequence of characters.

```
String s = "I love Java";  
System.out.println(s.contains("Java"));
```

Output: true

12. startsWith()

Checks whether the String starts with the specified text.

```
String s = "Java Programming";  
System.out.println(s.startsWith("Java"));
```

Output: true

13. endsWith()

Checks whether the String ends with the specified text.

```
String s = "Java Programming";  
System.out.println(s.endsWith("ing"));
```

Output: true

14. indexOf()

Returns the index of the first occurrence of a character or substring. Returns -1 if it is not found.

```
String s = "banana";  
System.out.println(s.indexOf("na"));
```

Output: 2

15. lastIndexOf()

Returns the index of the last occurrence of a character or substring.

```
String s = "banana";  
System.out.println(s.lastIndexOf("na"));
```

Output: 4

16. replace()

Replaces every occurrence of a character or matching character sequence.

```
String s = "Java is easy";  
System.out.println(s.replace("easy", "powerful"));
```

Output: Java is powerful

17. concat()

Joins one String to another String.

```
String a = "Hello";  
String b = "World";  
System.out.println(a.concat(" " + b));
```

Output: Hello World

18. isEmpty()

Returns true if the String has zero characters.

```
String s = "";  
System.out.println(s.isEmpty());
```

Output: true

19. isBlank()

Returns true if the String is empty or contains only whitespace. Available in modern Java versions.

```
String s = " ";  
System.out.println(s.isBlank());
```

Output: true

20. split()

Splits a String into an array using a delimiter or regular expression.

```
String s = "Java,Python,C";  
String[] parts = s.split(",");  
System.out.println(parts[0]);  
System.out.println(parts[1]);  
System.out.println(parts[2]);
```

Output:

Java

Python

C

21. String Concatenation with +

The + operator can be used to combine Strings and values of other data types.

```
String name = "Srinivas";  
int year = 2;  
  
System.out.println("Name: " + name);  
System.out.println("Year: " + year);
```

Output:

Name: Srinivas
Year: 2

22. Important: String Immutability

String methods do not modify the original String. They return a new String when a changed value is needed.

```
String s = "hello";  
s.toUpperCase();  
  
System.out.println(s);
```

Output: hello

To keep the changed value, assign the result: **s = s.toUpperCase();**

Quick Reference Table

Method	Purpose
length()	Number of characters
charAt()	Character at an index
substring()	Extracts part of a String
equals()	Content comparison
equalsIgnoreCase()	Case-insensitive comparison
compareTo()	Lexicographic comparison
toUpperCase()	Converts to uppercase
toLowerCase()	Converts to lowercase
trim()	Removes leading/trailing whitespace
contains()	Checks for text
startsWith()	Checks starting text
endsWith()	Checks ending text
indexOf()	Finds first occurrence
lastIndexOf()	Finds last occurrence
replace()	Replaces matching text
concat()	Joins Strings
isEmpty()	Checks zero length
isBlank()	Checks empty/whitespace-only
split()	Splits into an array

Tip: Practice these methods with different inputs and indexes. Pay special attention to 0-based indexing and the difference between `equals()` and `==` when comparing Strings.